

PocketLLM Portal Requirements Report

1. Context & Scope

- **System Name:** PocketLLM Portal
- **Purpose:** Lightweight web portal for interacting with a CPU-friendly LLM (Local Language Model).
- **Architecture:** React frontend + Fastify backend + SQLite cache.
- **Goal:** Deliver efficient, portable, and dependable LLM access for local devices.

2. Stakeholders & Personas

- **End User:** Chat interface through the browser.
- **Administrator:** Monitors health and metrics.
- **Developer:** Integrates API endpoints.
- **Instructor/TA:** Evaluates architectural and quality compliance.

3. Assumptions & Constraints

- **Hardware:** 2–6 vCPU, ≤16 GB RAM (tested on i7-9750H, 16 GB).
- **Runtime:** Node v22.16.0, npm 10.9.2, SQLite.
- **Network:** Localhost environment.
- **Model:** CPU-only lightweight LLM or mock generator.
- **Non-goals:** GPU support, fine-tuning, production deployment.

4. Functional Requirements (FR)

- **Prompt submission:** User enters text and sends to backend.
- **Single generation:** POST /api/v1/generate returns a full completion.
- **Streaming mode:** GET /api/v1/stream?prompt= streams tokens (SSE).
- **Cache mechanism:** Check/store completions by prompt hash.
- **Cache maintenance:** POST /api/v1/cache/sweep clears entries.
- **Metrics tracking:** /api/v1/admin/metrics exposes latency, counts, hit-rate.
- **Metrics reset:** POST /api/v1/admin/metrics/reset.
- **Health endpoint:** /api/v1/admin/health returns OK with timestamp.
- **Error handling:** Return structured JSON errors.
- **CORS support:** Allow defined origins only.
- **Configuration:** Environment-based ports and cache TTL.
- **UI feedback:** Loading indicator, token stream visualization.
- **Accessibility:** Keyboard operation, ARIA live regions.
- **Logging:** Structured logs for requests/responses.

5. Non-Functional Requirements (NFR)

Efficiency

- - Single (MISS): ≤ 1600 ms (Observed 1376.9 ms)
- - Single (HIT): ≤ 5 ms (Observed 0.207 ms)
- - Streaming TTFT: ≤ 50 ms (Observed 0.209 ms)
- - Streaming Total: ≤ 1600 ms (Observed 1383.0 ms)

Scalability

- Support 10 concurrent users with $\leq 20\%$ p95 increase.
- Maintain rolling metrics for ≤ 5000 samples.

Portability

- Compatible with Windows, macOS, Linux (Node + SQLite).
- Configurable via environment variables.

Adaptability

- Switch model or cache backend without code changes.
- Adjustable TTL and cache size.

Dependability

- Health uptime $\geq 99\%$ per hour.
- Graceful degradation on cache failure.
- Process never crashes on malformed input.

Security

- CORS enforced to frontend origin.
- Max request size 8 KB.
- No sensitive data in logs.

Usability

- First-time interaction within 10 seconds of load.
- WCAG AA color contrast and ARIA support.

Maintainability

- Zero lint/type errors.

- $\geq 70\%$ code coverage.
- Clear module separation.

Observability

- Metrics API exposes p50/p95/p99 latency, counts, cache hit rate.
- JSON structured logs per request.

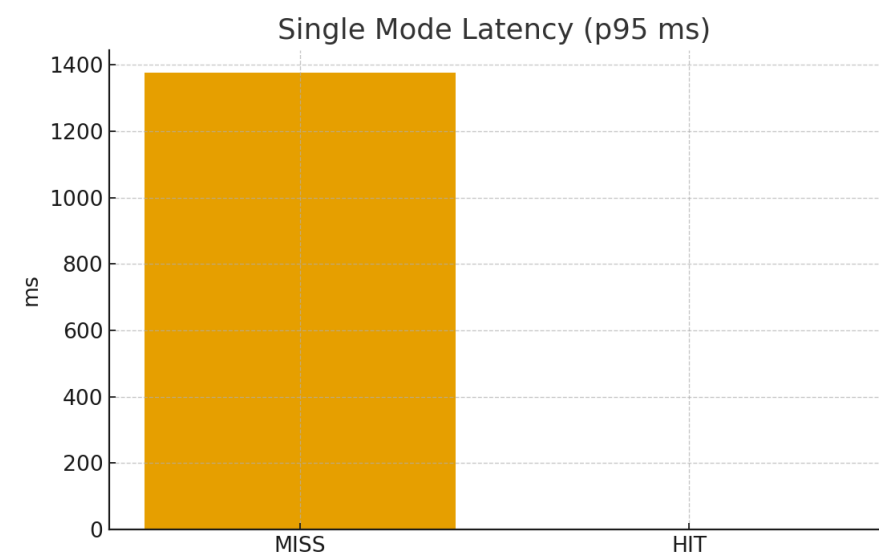
Complexity

- Function length ≤ 60 lines.
- Nesting depth ≤ 3 .
- No cyclic dependencies.

6. Measured Results

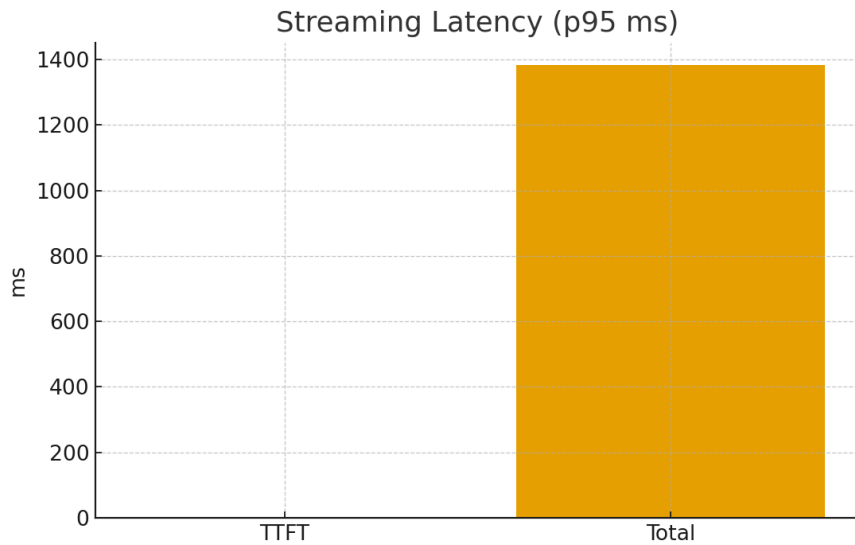
Single Mode Latency

Scenario	p50 (ms)	p95 (ms)	p99 (ms)
MISS	688.5	1376.9	1376.9
HIT	0.096	0.207	0.214

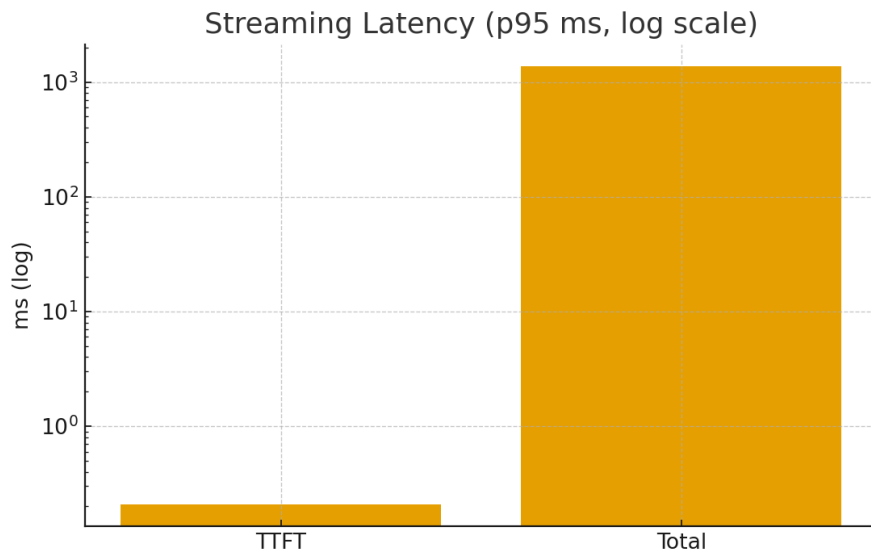


Streaming Latency

Metric	p50 (ms)	p95 (ms)	p99 (ms)
TTFT	0.083	0.209	0.209
Total	687.1	1383.0	1383.0



Streaming (log scale)



Environment

- **CPU:** Intel i7-9750H, 6C/12T
- **RAM:** 16 GB
- **OS:** Windows 11 (64-bit)
- **Node:** v22.16.0
- **NPM:** 10.9.2

7. API Contract

Endpoint	Method	Description
/api/v1/generate	POST	Generate completion; returns JSON.
/api/v1/stream	GET	Streams response via SSE.

/api/v1/admin/health	GET	Returns status + timestamp.
/api/v1/admin/metrics	GET	Returns latency stats, hit rate.
/api/v1/admin/metrics/reset	POST	Resets metrics.
/api/v1/cache/sweep	POST	Clears cache entries.

8. Architecture Views

- **Component Diagram:** Frontend ↔ Backend ↔ Cache/Model.
- **Sequence (Single):** UI → API → Cache → Model → Cache → Response.
- **Sequence (Stream):** UI → API → SSE stream → Tokens → End.
- **Deployment:** Browser client, Node server, SQLite file DB.

9. Risks & Mitigation

Risk	Mitigation
CPU bottleneck	Cache hits + token streaming reduce load.
DB contention	Single-writer mode, short transactions.
SSE breakage	Localhost test; keep-alive headers.

10. Future Enhancements

- Authentication & sessions.
- Model selection via UI.
- Distributed cache (e.g., Redis).
- GPU acceleration.

11. Sources

- USC CSCI 578 lecture notes (Weeks 1–5).
- Fastify, SQLite, and React official documentation.
- PocketLLM and local LLM CPU runner reference implementations.

12. Appendix

- **Raw JSON Files:** metrics_single_miss.json, metrics_single_hit.json, metrics_stream.json, environment.json.
- **Merged CSV:** pocketllm_nfr_merged.csv.
- **Charts:** chart_single_miss_vs_hit.png, chart_stream_ttf_tvs_total.png, chart_stream_ttf_tvs_total_log.png.
- **Run scripts:** A_metrics_install.ps1, B_run_measurements.ps1.
- **Config:** .env.sample for port, TTL, model path.